



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Profesor: Dr. Rigoberto Cerino Jiménez

Reto semanal 5. Manchester Robotics

Equipo *I miss NWJNS*

Tania Sofia Arrazola Bello	A01738124
Iñaki Ahedo Madrid	A01738231
Marcos Allen Martínez Cortés	A01737939

17 de mayo de 2026

Resumen

En el presente *Reto semanal 5*, se desarrolló un sistema de navegación autónoma para el robot Puzzlebot bajo ROS 2, integrando percepción visual y control distribuido. El sistema rastrea trayectorias estimando el error lateral (comunicado mediante `/puzzlebot/line_error`) y reacciona a tres estados semafóricos: Rojo (paro total), Amarillo (desaceleración) y Verde (marcha normal).

El procesamiento de visión, implementado con OpenCV, opera en paralelo. Para la pista, emplea ajuste de iluminación, binarización de Otsu y un análisis geométrico mediante scanlines delimitadas por una Región de Interés (ROI). Simultáneamente, el semáforo se identifica con un `SimpleBlobDetector` optimizado por color, circularidad (mínimo 0.72) y área, utilizando el tamaño del blob para estimar la proximidad y validar el estado a 35 cm de distancia.

El sistema de control está gobernado por un PID angular estabilizado con un filtro de Media Móvil Exponencial (EMA). Para optimizar el paso por curvas, se implementó un esquema de programación de ganancias (*gain scheduling*) que activa una constante proporcional agresiva (`KP_TURN`) ante errores elevados. Finalmente, la seguridad recae en una máquina de estados: al detectar luz roja, el estado `waiting_for_green` anula la velocidad y bloquea el avance, garantizando que el robot solo reanude su marcha al percibir una señal verde explícita.

Objetivos

El objetivo principal de este proyecto es desarrollar un sistema de navegación autónoma basado en el seguimiento de líneas en lazo cerrado para el robot Puzzlebot utilizando ROS 2. El sistema integra un controlador PID y una capa de percepción visual de doble propósito para ejecutar el rastreo de trayectorias con alta precisión y robustez. Para alcanzar este propósito, se establecieron los siguientes objetivos específicos:

- **Percepción y Estimación de Error.** Implementar un nodo de visión artificial que estime en tiempo real el error lateral del robot respecto al centro del carril, mediante técnicas de preprocesamiento, binarización de Otsu y un análisis geométrico apoyado en líneas de escaneo horizontales (*scanlines*).
- **Estabilización y Filtrado de Señal.** Diseñar una arquitectura de acondicionamiento de datos empleando una Región de Interés (ROI) y un filtro de Media Móvil Exponencial (EMA), encargados de mitigar el ruido del sensor óptico y evitar oscilaciones bruscas en las lecturas.
- **Control de Lazo Cerrado.** Desarrollar un controlador para regular el ángulo de dirección y modular la velocidad lineal, garantizando la corrección continua del rumbo y el seguimiento preciso de la pista mediante la publicación de mensajes de tipo Twist en el tópico `/cmd_vel`.
- **Navegación Condicionada por Visión.** Desarrollar un sistema de visión computacional paralelo, basado en detección de blobs, para la identificación cromática y estimación de proximidad de un semáforo, estableciendo una

jerarquía de seguridad: detención total y bloqueo (Rojo), velocidad reducida (Amarillo) o continuación de la marcha (Verde).

- **Gestión de Datos mediante ROS 2.** Administrar de manera eficiente el flujo de información y estados del sistema a través de los tópicos asegurando una comunicación síncrona y robusta entre los nodos de percepción y el módulo de control.
- **Optimización y Robustez del Sistema.** Garantizar la estabilidad y fiabilidad de la navegación frente a perturbaciones externas y ruido sensorial, integrando técnicas de estabilización de imagen y un filtro de Media Móvil Exponencial (EMA) en el lazo de control, lo que permite al sistema tolerar pérdidas momentáneas de la trayectoria y adaptarse a condiciones variables de iluminación sin comprometer su desempeño, así mismo evitar pérdida de línea integrando un sistema de control robusto adaptado para evitar cambios en el sistema.

Introducción

El presente reto aborda el diseño y la implementación de un sistema de navegación autónoma para un robot móvil basado en el framework ROS 2, fundamentado en un esquema de control en lazo cerrado y estimación del error mediante visión computacional (seguimiento de línea), además del reconocimiento de un semáforo operando en paralelo. El objetivo central es el rastreo continuo de una trayectoria física preestablecida, empleando un controlador PID sintonizado para corregir el ángulo de dirección del robot en tiempo real a partir de la estimación de su desplazamiento lateral, y adecuando su comportamiento cinemático a las restricciones dictadas por el estado del semáforo.

Control en lazo cerrado para un robot móvil

El sistema opera bajo un esquema de Control en lazo cerrado. A diferencia del lazo abierto, este método utiliza la retroalimentación constante de la pose estimada para comparar la posición actual con la meta deseada. Es decir, que utiliza la información del estado actual del robot para ajustar sus acciones y alcanzar un objetivo deseado [1]. A diferencia del lazo abierto este sistema detecta y corrige errores generados por perturbaciones externas, ruido en los sensores, o incertidumbres del modelo como el deslizamiento de las ruedas.

El funcionamiento se basa en un ciclo continuo que puede clasificarse en estas etapas [1, 2, 3]:

1. **Estimación y Percepción de la Pose:** El robot utiliza sensores como el IMU para la orientación y determinar su posición real (x , y , θ) así como también encoders para la velocidad de las ruedas [1].

2. **Cálculo del error:** Se compara la posición real con la trayectoria de referencia (el camino deseado). La diferencia entre ambas se denomina *error de seguimiento*. El error de seguimiento es la medida de la desviación entre el estado actual del robot y el estado deseado definido por la trayectoria. En este proyecto, el error no se calcula de forma global, sino que se divide en dos componentes polares para facilitar el control del robot diferencial:

- a. *Error de Distancia (Rho):* Se define como magnitud del vector que une la posición actual del robot (x, y) con la coordenada objetivo (x_{goal}, y_{goal}), esto representando cuanto camino lineal le falta recorrer al robot. La fórmula es la siguiente:

$$\rho = \sqrt{(x_{goal} - x)^2 + (y_{goal} - y)^2}$$

- b. *Error de Orientación (Alpha):* Es la diferencia angular entre el ángulo de vista del robot y el ángulo necesario para apuntar directamente hacia el objetivo, este error indica al robot cuánto tiene que girar sobre su eje para lograr el objetivo. La fórmula es la siguiente:

$$\alpha = \text{atan2}(y_{goal} - y, x_{goal} - x) - \theta$$

3. **Acción de Control:** Un controlador procesa el error y genera comandos de velocidad para los motores que minimicen la desviación.

Robustez de un controlador

La robustez de un controlador se basa principalmente en su capacidad para garantizar el cumplimiento de un objetivo de control, seguir una trayectoria, ante la presencia de perturbaciones externas, ruidos en los sensores e incertidumbres físicas no modeladas [1]. Los principales puntos a tomar en cuenta para un sistema robusto son:

1. **Manejo de Incertidumbres y Perturbaciones.** Un controlador robusto debe ser capaz de minimizar el impacto de factores que no están incluidos en el modelo matemático original, tales como el deslizamiento de las ruedas, no linealidades y ruidos e imperfecciones del entorno [1].
2. **Estrategias de Control Específicas.** Existen algunas técnicas para el diseño que implementan un control robusto como lo son:
 - a. *Control en Modo Deslizante (SMC):* Es una de las técnicas más citadas por su robustez. Se basa en forzar al sistema a permanecer sobre una superficie deslizante predefinida [1]. Una vez que el sistema alcanza este estado, adquiere características invariantes ante perturbaciones.
 - b. *Control de Espacios Nulos (Hierarquización):* Proporciona robustez al establecer una jerarquía de tareas donde la tarea principal, como la posición, se ejecuta sin interferencia de las tareas secundarias, como la orientación, proyectando estas últimas en un espacio que no afecta la precisión del objetivo primordial [1].

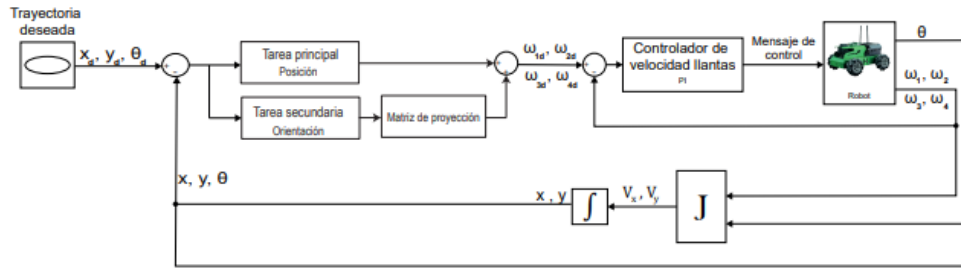


Figura 1: Diseño de un controlador usando lazo de control en espacios nulos [1].

3. **Arquitectura de Lazo Cerrado y Sintonización.** Para que un controlador, como el PID, sea robusto, sus ganancias deben ajustarse cuidadosamente considerando las perturbaciones y ruidos específicos del entorno de operación. Además, para mejorar la robustez frente a sensores ruidosos, se utilizan observadores, como los basados en la técnica *Super Twisting*, que permiten obtener estimaciones de velocidad más limpias y estables que la simple derivación de la posición [2]. Por último, la implementación de una estructura de doble lazo, con un lazo interno de velocidad, asegura que las ruedas mantengan la consigna deseada incluso ante obstáculos físicos como pendientes, haciendo que el sistema sea resistente a fallos mecánicos menores [1].

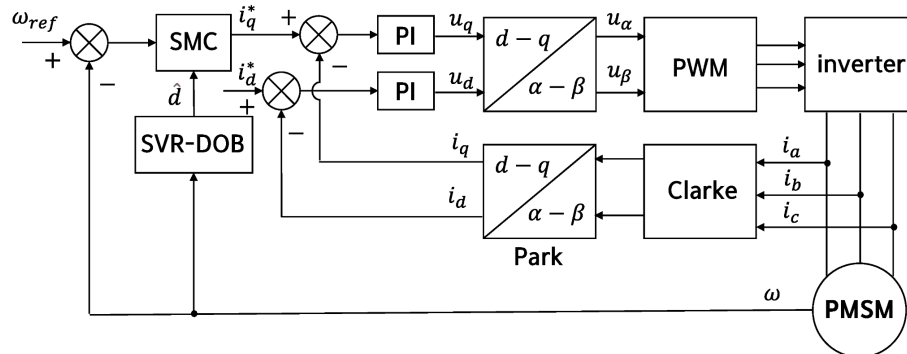


Figura 2: Diseño de un controlador usando la técnica super twisting, PI y SMC para mejorar la robustez. <https://www.mdpi.com/2076-3417/12/21/10749>

Cinemática de robots con accionamiento diferencial

Es la representación matemática que describe cómo el movimiento de sus dos ruedas independientes se traduce en el desplazamiento y la rotación del vehículo [4]. En el caso del Puzzlebot, se utiliza un sistema de accionamiento diferencial, el cual consiste en dos ruedas motrices independientes situadas en un mismo eje y una rueda loca para brindar estabilidad. Este modelo relaciona las velocidades angulares de cada rueda con la velocidad lineal y angular total del robot basándose en su geometría, específicamente en el radio de las ruedas (r) y la distancia entre ellas (L) [4]. El funcionamiento de este tipo de robots se divide en dos enfoques principales:

- **Cinemática Directa (Forward Kinematics):** Permite calcular la velocidad lineal v_x y la velocidad angular w del robot a partir de las velocidades de rotación medidas de la rueda derecha y de la rueda izquierda
- **Cinemática Inversa (Inverse Kinematics):** Se utiliza para determinar qué velocidades deben tener las ruedas para que el robot alcance una velocidad lineal y angular deseada

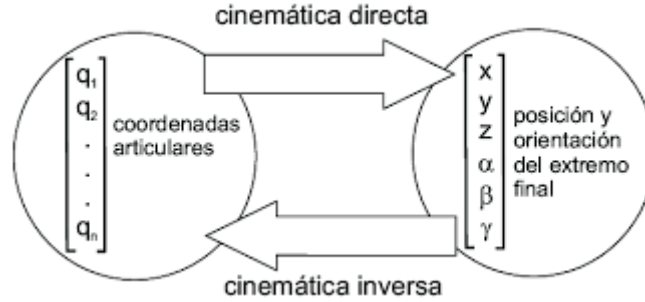


Figura 3: Relacion entre la cinemática directa e inversa

https://www.researchgate.net/figure/Figura-216-Relacion-entre-cinemática-directa-e-inversa_fig15_287995531

El modelo se divide en dos partes:

1. *Cálculo de velocidades lineales y angulares.* La velocidad lineal total v del centro del robot y su velocidad angular w se obtienen mediante las siguientes ecuaciones:

$$\omega = (v_R - v_L)/b \quad V = \omega \cdot R = \frac{v_R + v_L}{2} \quad (1)$$

Donde b es el ancho de la vía o distancia entre las ruedas (wheelbase) [5].

2. *Representación en coordenadas globales.* Para conocer la posición del robot en cada instante, se transforma el movimiento del cuerpo al sistema de coordenadas global utilizando el ángulo de orientación actual ϕ [5].

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \varphi & 0 \\ \sin \varphi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} V \\ \omega \end{bmatrix} \quad (2)$$

Procesamiento de imágenes en una tarjeta embebida

Se denomina *Visión Embebida* al procesamiento de imágenes en tarjetas embebidas y consiste en integrar algoritmos en hardware dedicado (circuitos integrados, GPUs o FPGAs) que transforman datos visuales en decisiones en tiempo real [6]. El reto principal es ejecutar tareas computacionales pesadas, tales como la detección de bordes, la segmentación y la clasificación de imágenes, en entornos con recursos limitados de memoria y energía [6, 7].

Para implementar estas soluciones, se requiere de un ecosistema que equilibre flexibilidad y potencia, conformado por:

- **Software.** Utilizando la librería de código abierto OpenCV y Python por la gran comunidad que contribuye al desarrollo de aplicaciones [6].
- **Sistemas Operativos.** Linux es el estándar cuando se trata de estabilidad y seguridad y permite mayor flexibilidad que un sistema operativo tradicional [6].
- **Controladores y sensores.** Los microcontroladores permiten un rápido prototipado para la interacción con el entorno físico, pero para videos de alta resolución se necesitan módulos más potentes [6].

De este modo, ante la exigencia del procesamiento en tiempo real, la arquitectura del hardware resulta crítica:

- **GPUs (Unidades de Procesamiento Gráfico).** Módulos como los de NVIDIA (ej. Tegra o Cortex A57) son apropiados para ejecutar en paralelo múltiples redes neuronales y procesar video con baja latencia [6, 7].
- **FPGAs y DSPs.** Estas tarjetas permiten un procesamiento en paralelo masivo y directo, a través de hardware, que elimina los cuellos de botellas típicos de los buses seriales convencionales. Esto asegura que la latencia sea de microsegundos [7].
- **Optimización de algoritmos.** Se utilizan técnicas como la cuantización (disminución de la precisión de los datos) y la poda de redes neuronales para que modelos de IA complejos puedan caber en la RAM de la tarjeta [7].

El rendimiento se mide en términos del throughput (píxeles procesados por segundo) y de la latencia; así, la tendencia actual es reducir el ancho de banda mediante el procesamiento selectivo (solo analizar áreas visualmente significativas) y la integración de modelos de deep learning eficientes, permitiendo que las tarjetas embebidas operen de forma autónoma, económica y confiable en sectores como la robótica y el automotriz [7].

Interconexión entre la Jetson y la cámara

La NVIDIA Jetson Nano es la unidad de procesamiento principal del sistema, encargada de tareas de alto nivel como: ejecutar ROS 2, procesar datos, implementar algoritmos complejos, visión por computadora, etc. Cuenta con mayor capacidad de cómputo que la Hackboard, lo que permite ejecutar procesos más complejos aunque con menor prioridad en tiempo real. La Raspberry Pi Camera es el sensor visual del Puzzlebot utilizado principalmente para: capturar imágenes y video, aplicaciones de visión por computadora, detección de objetos, etc. [8]

La Jetson Nano permite usar cámaras de alta velocidad gracias a su soporte nativo para el protocolo MIPI-CSI (Mobile Industry Processor Interface - Camera Serial Interface), que conecta la cámara directamente al procesador sin la latencia del USB. A diferencia de los modelos anteriores que requieren configuraciones complejas de drivers, la Jetson

Nano viene con un conector compatible y drivers preinstalados para la Raspberry Pi Camera V2 (sensor Sony IMX219). Esto permite conectar directamente a la Jetson Nano mediante interfaz CSI, permitiendo el procesamiento en tiempo real de las imágenes [9].



Figura 4: Conexión física de la Jetson Nano con la Raspberry Pi Camera V2.
https://cdn1.botland.store/img/art/inne/15751_5a.jpg

Visión por computadora

La visión por computadora es un conjunto de técnicas que permiten extraer información de imágenes, video o nubes de puntos, e incluye técnicas basadas en *deep learning*, técnicas basadas en detección y extracción de características, técnicas de procesamiento de nubes de puntos, técnicas de procesamiento de visión en 3D, y, el procesamiento de imágenes que suele aplicarse como preprocesamiento en la visión por computadora y que comprende técnicas como aumento de datos, conversión o corrección de color, redimensionamiento de imágenes, eliminación de ruido, entre otras [10].

La visión por computadora es importante para una amplia gama de aplicaciones del mundo real, incluyendo a los sistemas autónomos que utilizan varios sensores que recopilan datos visuales del entorno, los cuales son utilizados para segmentar carreteras, senderos o edificios, así como para detectar personas y vehículos [10].

Métodos de procesamiento de imágenes y visión por computadora

- **Espacios de color y conversión.** Los espacios de color son organizaciones específicas que determinan cómo los colores de las imágenes digitales son representados y procesadas en visión computacional, y son esenciales para aplicaciones como la detección de objetos, segmentación, corrección de color y compresión; un espacio de color define el modelo de color y la función de mapeo abstracto utilizada para definir los colores reales [11].

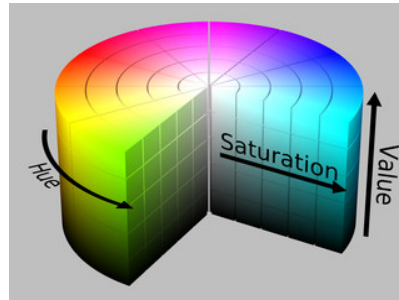


Figura 5: Representación tridimensional del espacio de color HSV [11].

En OpenCV, la conversión de espacios de color se realiza con la función `cvtColor()`, que permite la transformación de imágenes entre espacios de color como RGB y HSV. El espacio RGB es el más utilizado comúnmente, mientras que otros como el HSV (Hue, Saturation, Value) permite detectar y filtrar colores con mayor facilidad, incluso cuando la iluminación cambia, ya que es más cercano a cómo los humanos representan los colores [11].

- **Ajuste de brillo y contraste.** Ajustar el brillo y contraste de una imagen puede ayudar a corregir defectos en las imágenes y facilita la visualización de detalles. Existen diversas maneras de ajustar el brillo y contraste con OpenCV, dos de ellas son las funciones `addWeighted()` y `convertScaleAbs()`, que ajustan el brillo al añadir un valor escalar a cada pixel, y el contraste al escalar el valor de los pixeles [12].

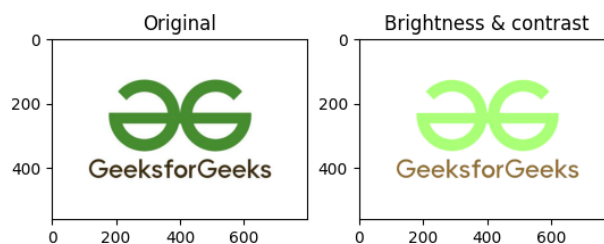


Figura 6: Comparación antes y después del ajuste de brillo y contraste [12].

- **Desenfoque Gaussiano.** El desenfoque Gaussiano es una técnica de filtrado que utiliza una función gaussiana para crear un efecto de suavizado en una imagen, este suavizado reduce el ruido de la imagen y el detalle. En OpenCV, puede utilizarse con la función `GaussianBlur()`, que toma parámetros como la imagen, el tamaño del kernel, la desviación estándar para las direcciones X y Y, y `borderType` [13].

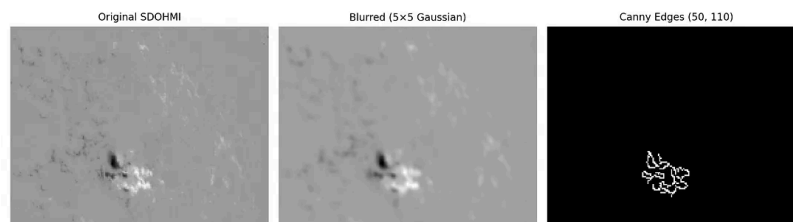


Figura 7: Comparación sobre el uso del desenfoque Gaussiano [13].

- **Detección de blobs (regiones).** La detección de blobs es una técnica que permite detectar blobs, que son regiones de una imágenes con propiedades específicas tales como intensidad o color uniformes o dentro de un rango definido; es decir, grupos de píxeles conectados entre sí que se consideran una unidad de información. En OpenCV, la detección de blobs se realiza mediante `SimpleBlobDetector`, que es un algoritmo que sigue los siguiente pasos: umbralización, agrupación, unión (merging) y cálculo del centro y del radio. Las técnicas para detección de blobs incluyen el Laplaciano de Gauss (LoG), Diferencia de Gaussianas (DoG) y Determinante de Hessiano (DoH) [14].

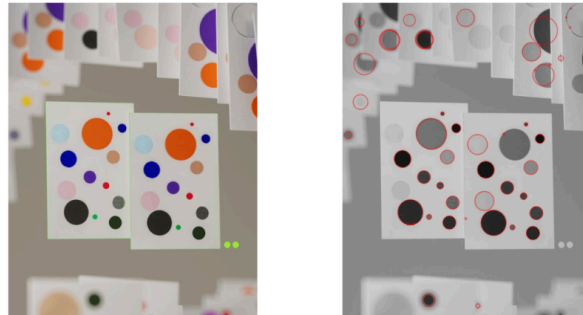


Figura 8: Ejemplo de la detección de blobs [14].

En OpenCV, los parámetros determinan los filtros para la detección de blobs. Entre estos se encuentra detección por: color, tamaño, forma, circularidad, convexidad y radio inercial. Entre estos, el filtrado por color determina si se filtran los blobs oscuros o claros, y el filtrado por circularidad utiliza una medición que determina qué tan cercano es el blob a un círculo [14].

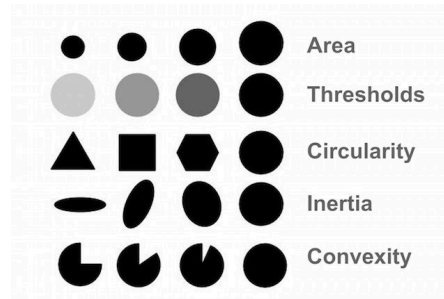


Figura 9: Ilustración de los parámetros del detector de blobs [14].

- **Operaciones morfológicas.** Las operaciones morfológicas son un conjunto extenso de operaciones que se utilizan para procesar imágenes con base en figuras geométricas. Las operaciones morfológicas, al implementar un elemento estructurante a una imagen de entrada, generan otra imagen de salida que tiene el mismo tamaño. La erosión y la dilatación son las operaciones morfológicas más elementales. La erosión elimina píxeles de los bordes de los objetos, a la vez que la dilatación añade píxeles a dichos límites. La cantidad de píxeles que se eliminan o se añaden a los objetos está determinada por la forma y el tamaño del elemento estructurante empleado para el procesamiento [15].

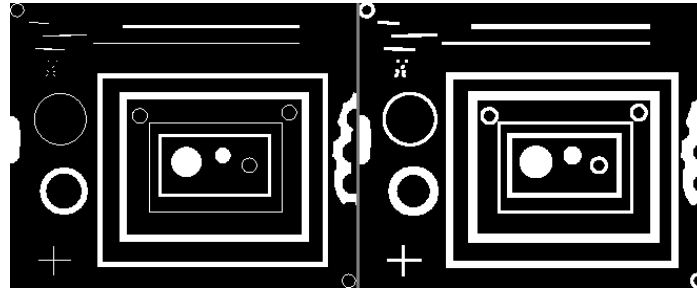


Figura 10: Ejemplo de la dilatación, imagen original y procesada [15].

Algoritmo Scanlines

El algoritmo original de *Scanline* se diseñó como una técnica para rellenar polígonos con color en una pantalla. En lugar de procesar píxel por píxel de manera aleatoria, este algoritmo escanea la imagen de izquierda a derecha, una sola línea horizontal (*scanline*) a la vez. Mientras hace este barrido, lleva un registro, llamado "Active Edge List" o AEL, de los bordes del polígono que intersectan con esa línea horizontal actual [16]. Una vez que encuentra los puntos de intersección, el borde izquierdo y el borde derecho del polígono en esa fila específica, el algoritmo rellena de color todos los píxeles que se encuentran entre esos dos puntos [16].

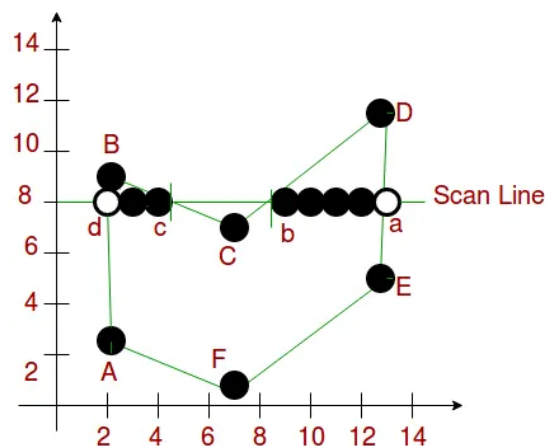


Figura 11: Ejemplo del algoritmo scanline.

<https://medium.com/@dillihangrae/scanline-filling-algorithm-852ad47fb0dd>

Existen casos especiales que deben considerarse cuando la línea de escaneo (*scanline*) intersecta los vértices de un polígono:

- **Intersecciones del Mismo Lado:** Si ambas líneas que convergen en el vértice se encuentran del mismo lado de la línea de escaneo, el vértice se trata como dos puntos [17].
- **Intersecciones en Lados Opuestos:** Si las líneas que se intersectan en el vértice se encuentran en lados opuestos de la línea de escaneo, el vértice se considera como un solo punto [17].

Estructuras de datos del Algoritmo de Relleno de Polígonos por Línea de Escaneo (*Scanline*)

El algoritmo de línea de escaneo consta de varias estructuras de datos que trabajan en conjunto para rellenar el polígono. Estos componentes son:

- **Contenedores de Aristas (Edge Buckets):** Los contenedores de aristas almacenan datos sobre cada borde del polígono. Contienen detalles importantes como la coordenada y máxima, la coordenada x del punto más bajo y la pendiente inversa de la arista [17].
- **Tabla de Aristas (Edge Table - ET):** La tabla de aristas contiene todos los bordes del polígono organizados en listas. Cada lista corresponde a una línea de escaneo específica, y las aristas dentro de la lista se ordenan en función de sus valores yMin (la y mínima de la arista). La tabla de aristas ayuda a organizar los bordes para que puedan procesarse de manera eficiente, línea de escaneo por línea de escaneo [17].
- **Lista Activa (Active List - AL):** La lista activa es la encargada de llevar el registro de las aristas actuales que se están utilizando para rellenar el polígono en un momento dado. Las aristas se añaden a la lista activa desde la tabla de aristas cuando su valor yMin coincide con la línea de escaneo actual. La lista activa se actualiza y se vuelve a ordenar después de procesar cada línea de escaneo [17].

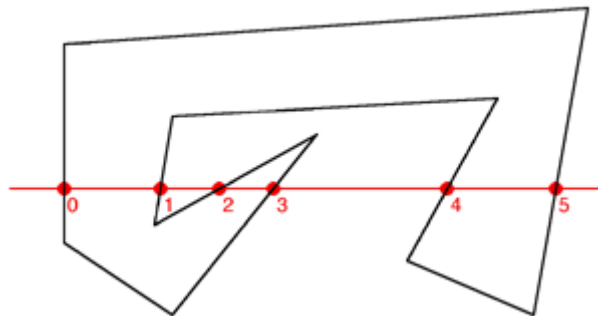


Figura 12: Ejemplo del algoritmo scanline. https://alienryderflex.com/polygon_fill/

Robustez en sistemas de procesamiento de imágenes

La robustez en la visión artificial se define como la capacidad de un algoritmo para mantener su integridad operativa y precisión estadística ante la presencia de perturbaciones externas y variaciones en el entorno de captura. En el contexto de la robótica autónoma, este concepto es fundamental, ya que los sensores visuales están intrínsecamente sujetos a ruidos térmicos, cambios abruptos en la iluminación y oclusiones dinámicas [18].

Uno de los pilares de la robustez visual es la selección de un espacio de color que minimice la dependencia de la intensidad de la luz. Mientras que el modelo RGB es dependiente de la luminancia, el espacio HSV (Hue, Saturation, Value) permite una

separación efectiva entre la información cromática (matiz) y la iluminación (brillo). Según Gonzalez y Woods (2018), esta propiedad es esencial para el diseño de clasificadores robustos, ya que el matiz permanece relativamente constante bajo diferentes condiciones de exposición, facilitando una segmentación de objetos más fiable en entornos no controlados [19].

Más allá del color, la robustez se refuerza mediante el análisis de la morfología geométrica. La implementación de filtros de circularidad, área y excentricidad permite al sistema discriminar entre el objeto de interés y el ruido ambiental. La teoría de análisis estructural sugiere que el uso de descriptores de forma basados en momentos de Hu o circularidad de compactación permite una identificación invariante a la escala y la rotación [20]. Estos filtros actúan como una capa de validación que asegura que las detecciones erráticas sean descartadas antes de ingresar a la lógica de control. Por otro lado, la robustez también está ligada a la eficiencia computacional. Un sistema robusto debe equilibrar la complejidad del procesamiento con la latencia del sistema de control. La integración de técnicas de filtrado morfológico, como la erosión y dilatación, ayuda a eliminar el ruido tipo "sal y pimienta" como menciona corke, permitiendo que la señal visual que alimenta al sistema de navegación sea limpia y persistente en el tiempo [21]. Esto es crítico para evitar comportamientos inestables en la navegación reactiva, donde una detección falsa podría traducirse en una respuesta motriz errónea.

Solución del problema

1. Nodo de detección de líneas

Se desarrolló un algoritmo determinado como *scanlines*, que determina la posición del robot con respecto a la línea que debe de seguir. Se creó un nodo llamado `line_detector` el cual hace el siguiente procedimiento:

- Se suscribe al tópico `/video_source/raw`, y a la imagen capturada desde dicho tópico se le ajusta el contraste y brillo para eliminar reflejos que pueda provocar la cinta que marca la línea. Para aquello se utilizó la función de `cv2.convertScaleAbs()` la cual se le mandó como argumento un contraste alpha y un brillo beta determinado.
- Después se convierte la imagen a escala de grises y se le aplica un filtro Gaussiano con un kernel de 7x7 para reducir aún más el ruido.
- Para la detección de la línea, se utilizó la técnica de umbralización de Otsu combinado con un umbral inverso para separar la línea oscura con el fondo café claro.
- Para evitar que las líneas laterales del carril a seguir provocan un cálculo del error incorrecto y evitar detecciones de otros objetos, se aplicó una región de interés (ROI), eliminando los bordes laterales de la imagen, limitando la visión de la cámara a únicamente ver la línea central entre los píxeles 190 y 450 en el eje X.

- Se utilizó el algoritmo de scanlines, tomando la misma filosofía de barrer la imagen horizontalmente para encontrar bordes, pero en lugar de usarlo para colorear figuras, se utilizó para encontrar el centro del carril por donde debe conducir el robot. Se muestra a continuación el funcionamiento:
 1. Para evitar escanear toda la imagen, en la mitad inferior de la imagen, se dibujan 5 líneas horizontales en los píxeles 390, 410, 430, 450, 475. Esta zona es donde físicamente se encuentra el camino justo frente al robot.
 2. Para cada una de esta 5 líneas el algoritmo lee píxeles de izquierda a derecha buscando los de color blanco que representan la línea de la pista tras la binarización.
 3. Igual que el algoritmo de relleno encuentra un borde izquierdo y derecho, el robot detecta el primer píxel blanco, el borde izquierdo del carril, y el último píxel blanco de esa fila, el borde derecho, luego, los promedia $(p_left + p_right) / 2.0$ para calcular el centro exacto de la línea blanca en esa altura de la imagen.
 4. Finalmente, el error de posición es determinado por la diferencia entre el centroide calculado de la línea y el centro de la imagen. Se promedian los errores de los 5 scanlines detectados y se publica el resultado para el nodo de control mediante el tópico de `/puzzlebot/line_error`.

2. Nodo del controlador

Para que el Puzzlebot sea capaz de seguir la línea detectada, se desarrolló un nodo de ROS 2 para el control, el cual se suscribe al tópico `/puzzlebot/line_error` y publica al tópico `/cmd_vel`.

A través de la función `control_callback`, se controla la velocidad necesaria para el movimiento del robot a través de un controlador PID discretizado. Dentro del callback, lo primero que se realiza es aplicar un filtro de Media Móvil Exponencial (EMA), para suavizar la entrada del controlador, que es el error de posición reportado por el nodo de detección de líneas. El filtro EMA es un filtro pasabajos que utiliza el valor actual, el valor pasado (ya filtrado) y un factor alpha de 0.75, que funciona como coeficiente de suavización; con dicho valor se le da mayor peso al valor actual que al pasado, y en código se utiliza de la siguiente manera:

```
self.ema_error = (EMA_ALPHA * current_error) + ((1.0 -
EMA_ALPHA) * self.ema_error)
```

Posteriormente, la entrada pasa por el controlador PID discretizado, el cual incluye un anti-windup para el término integral. Además, el controlador PID utiliza dos diferentes valores de K_p , uno para los tramos rectos y otro para los tramos curvos, los cuales se pueden diferenciar a través de un umbral del error, el cual determina que cuando el error tiene un valor mayor a 60, el robot se encuentra en un tramo de curva.

Posteriormente, se calcula la velocidad angular, siendo esta la salida del controlador PID, el cual se utiliza para la construcción de un mensaje tipo `Twist` que será publicado en el tópico `/cmd_vel`. A este mensaje se le asignan dos componentes:

- **Velocidad angular (en z).** La velocidad angular se calcula con el controlador PID, y con la función `clip()` de la librería `numpy`, se acorta dentro del rango `[-1, 1]` rad/s.
- **Velocidad lineal (en x).** La velocidad lineal se fija, con dos posibles valores dependiendo de si se encuentra en un tramo recto o en una curva; se asigna una velocidad lineal de 0.12 m/s si se encuentra en un tramo recto, y, en caso contrario se asigna una velocidad lineal de 0.07 m/s.

3. Sintonización del controlador

La calibración del controlador PID y de los nodos de percepción se llevó a cabo mediante un enfoque *heurístico* basado en pruebas empíricas iterativas. Para el esquema de navegación, se sintonizaron las ganancias y velocidades de forma diferenciada: se ajustaron valores conservadores para mantener la estabilidad del robot, evitando oscilaciones, en los tramos rectos (`KP`, `LINEAR_NORMAL`), y parámetros más agresivos para asegurar una respuesta rápida y evitar que perdiera la pista en las curvas (`KP_TURN`, `LINEAR_TURNING`).

De este modo, se obtuvieron los siguientes valores para las ganancias: `KP = 0.0015`, `KP_TURN = 0.004`, `KI = 0.0000`, `KD = 0.0075`; utilizando en la ejecución un controlador PD, debido a las ganancias sintonizadas.

4. Integración del controlador con la detección de semáforos

En el reto semanal 4, se había desarrollado un nodo de ROS 2 llamado `TrafficLightNode` diseñado para detectar el color de un semáforo (rojo, amarillo, verde) utilizando visión computacional con OpenCV. El nodo tiene un suscriptor al tópico `/video_source/raw` y tiene dos publicadores a los tópicos `/semaforo_estado` e `/image_processing/result`.

Dentro del callback, el preprocesamiento comienza con una reducción del brillo y un desenfoque gaussiano para suavizar el ruido; posteriormente, se cambia el espacio de color de BGR a HSV, para detectar colores específicos a través de la creación de máscaras binarias la función de OpenCV `inRange()`, centrándose principalmente en los valores del Hue, donde blanco representa el color detectado y el negro lo demás.

Una vez obtenidas las máscaras, se buscan blobs en las máscaras de rojo, amarillo y verde, filtrando adicionalmente por área y circularidad (mínimo 0.72), utilizando `SimpleBlobDetector` de OpenCV para filtrar el área correspondiente a la luz del semáforo. De los blobs detectados se selecciona el color que tenga el blob más grande, y posteriormente se aplica un umbral con valor de 80, que permite que la

detección del semáforo se aplique cuando la distancia respecto al robot sea de 30 cm o menos.

Finalmente, se publica en el tópico `/semaforo_estado`, un mensaje tipo `String` con la letra del estado final: R, A, V, o N; y en el tópico `/image_processing/result` se publica una imagen procesada para calibración en tiempo real y debugging.

Para la integración del nodo desarrollado con el controlador para la detección de líneas, primero se realizaron pruebas de calibración, para detectar si era necesario realizar modificaciones. Utilizando `rqt_image_view`, se observó la imagen publicada para debugging, y con esto se detectaron dos elementos a modificar:

- **Cambios en las máscaras de color.** Parte importante de la detección del semáforo son las máscaras de color, que permiten detectar correctamente el estado en el que se encuentra el semáforo. Y si bien, se implementaron estrategias de robustez en el procesamiento de la imagen, al probar el Puzzlebot en un entorno diferente con cambios en la pista e iluminación, se detectó que la máscara encargada de detectar el color amarillo no estaba operando correctamente. De este modo, se realizó un cambio en los rangos de la máscara, particularmente disminuyendo el valor mínimo de saturación a 0, lo que permite que la máscara de color opere cuando la luz se acerque más a blanco.
- **Cambios en el área del semáforo.** Debido a los requerimientos del reto semanal, el semáforo ya no fue posicionado al centro de la imagen, sino a un costado de la pista; tomando esto en cuenta, se realizó una calibración para determinar el valor de umbral que relacionado el área del semáforo con la distancia a la que se encuentra posicionado. Así se determinó que, colocando el semáforo al costado de la pista a una distancia de 35 cm en línea recta, el valor de umbral sería de 30 px para el área del semáforo.

Con estas modificaciones se integró el nodo `TrafficLightNode` al paquete de ROS 2. Se realizaron las modificaciones al nodo `LineController`, comenzando por añadir un suscriptor al tópico `/semaforo_estado`, y dentro de su callback, se implementó la lógica de estados del semáforo:

- **Estado R.** Cuando se detecta el semáforo en rojo, se detiene el Puzzlebot y se activa la bandera `waiting_for_green`, que permite que el robot se mantenga detenido incluso cuando se quite el semáforo.
- **Estado A.** Cuando se detecta el semáforo en amarillo, se asigna un `speed_multiplier` de 0.7, haciendo que el Puzzlebot desacelere.
- **Estado V.** Cuando se detecta el semáforo en verde, se desactiva la bandera `waiting_for_green`, siendo esta la condición que permite que el Puzzlebot reanude su marcha; además, se asigna un `speed_multiplier` de 1, regresando al Puzzlebot a su velocidad estándar.

- **Estado N.** Cuando no se detecta el semáforo, se evalúa la bandera `waiting_for_green`; en caso de que no esté activa, el Puzzlebot puede con su marcha a su velocidad estándar (`speed_multiplier` de 1).

Y, en el cálculo de las velocidades se añadió el `speed_multiplier`, lo que permitió adecuar el comportamiento del Puzzlebot al estado del semáforo.

5. Nodo de publicación de la cámara

Durante el desarrollo del reto semanal, se determinó que la creación de un nodo para publicar en tiempo real el video de la cámara, permitiría tener un mejor dominio de su funcionamiento y la capacidad de inspeccionar libremente cualquier error presentado.

Así, se desarrolló el nodo `CameraPublisher` que funciona a través del framework `GStreamer`, que permite leer la cámara a través de su interfaz serial; durante la configuración de los parámetros para `GStreamer`, se estableció que la resolución del video publicado en el tópico de ROS sería de 640 x 480 px a 30 FPS. Y, posteriormente, se realizó la conversión con `CvBridge()` para poder publicar el video en el tópico `/video_source/raw` mediante un mensaje tipo `Image` de ROS.

6. Creación de archivo launch

Para facilitar la ejecución del paquete de ROS 2 se desarrolló un archivo de lanzamiento (*launch*) para iniciar y configurar varios nodos de forma automática y coordinada. El archivo `full_system_launch.py`, se encarga del lanzamiento de 4 distintos nodos:

- 1) `CameraPublisher`.
- 2) `TrafficLightNode`.
- 3) `LineDetector`.
- 4) `Line Controller`.

Los nodos se activan en el orden secuencial presentado anteriormente, a través de un encadenamiento de eventos; con esto, la ejecución de cada uno está estrictamente condicionada al lanzamiento del anterior.

7. Estrategias de robustez implementadas

Para asegurar la robustez del Puzzlebot en este challenge, se aplicaron las siguientes estrategias dentro del código:

- 1) **Filtrado de Señal mediante Media Móvil Exponencial (EMA).** En el controlador de línea, se aplicó un filtro EMA (con un factor *alpha* de 0.75) para suavizar la señal de error lateral proveniente de la cámara. Esto atenúa los picos de ruido (*spikes*) generados por reflejos o falsas detecciones así evitando que el robot oscile violentamente.

- 2) ***Programación de Ganancias (Gain Scheduling)***. Se implementó un ajuste dinámico del control proporcional y de la velocidad lineal. Al superar un umbral de error, el sistema transiciona a un estado de curva, es decir, reduce la velocidad de avance y aplica una constante proporcional más agresiva. Esto asegura agilidad en curvas cerradas sin sacrificar la estabilidad y suavidad en tramos rectos.
- 3) ***Redundancia por Promedio de Scanlines***. En lugar de depender de una sola lectura, el algoritmo de visión promedia el error extraído de cinco líneas de escaneo a diferentes alturas. Esta redundancia geométrica permite que, si un segmento de la línea está borroso, interrumpido o sufre un destello de luz, las lecturas restantes compensen la anomalía para entregar un dato de error estable.
- 4) ***Delimitación por Región de Interés (ROI)***. Se aplicaron restricciones espaciales igualando a cero los píxeles ubicados en los extremos laterales del fotograma. Esta estrategia vuelve al sistema inmune al ruido visual del entorno enfocando el procesamiento exclusivamente en el área de conducción.
- 5) ***Acondicionamiento Lumínico***. Para combatir la susceptibilidad de la binarización de Otsu ante los cambios de luz ambiental, se integró una etapa de preprocesamiento que fuerza una reducción de brillo y contraste (`ALPHA_CONTRAST`, `BETA_BRIGHTNESS`). Acompañado de un filtro Gaussiano, esto mitiga el impacto de sombras marcadas y reflejos de iluminación artificial en el piso.
- 6) ***Validación del Semáforo por Geometría y Proximidad***. Para la detección de la luz, no se confió únicamente en máscaras de color. Se utilizó un `SimpleBlobDetector` exigiendo una circularidad estricta (mínimo 0.72) para descartar objetos del mismo tono en el fondo. Además, se acondicionó la reacción del robot al área del blob, lo que sirve como un estimador de proximidad robusto, evitando que el robot se detenga prematuramente por luces que aún están demasiado lejos.

Resultados

Después de haber realizado el algoritmo de detección de línea, se comprobó su funcionamiento mediante la herramienta de `rqt_image_view` de ROS2. En esta, se puede observar los tópicos de tipo `Image` que publica el nodo detector de líneas. Primero para saber si el funcionamiento de la detección de bordes mediante el método de Otsu, en la interfaz se suscribió al tópico `/puzzlebot/camera/thresh` en el cual se puede observar una imagen binarizada con la línea pintada de color blanco y la pista en color negro.

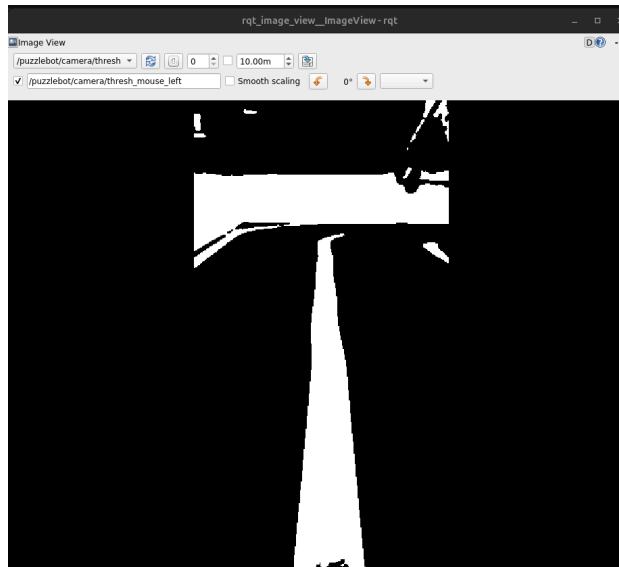


Figura 13: Imagen binarizada de la línea en una recta.

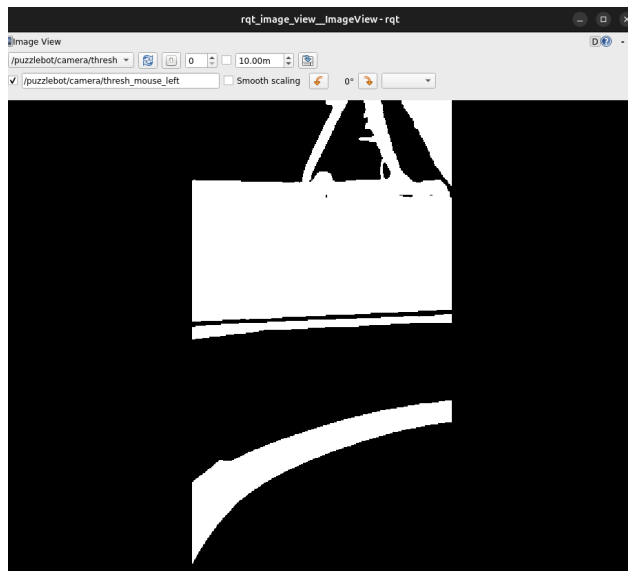


Figura 14: Imágen binarizada de la línea en una curva.

Como se observa en las figuras 13 y 14, la línea central se detecta con claridad, determinando que las variables del preprocesamiento como el kernel del blur Gaussiano, el valor del contraste y brillo y la binarización son las adecuadas para evitar ruido y tener una imagen limpia de la línea.

Para comprobar el funcionamiento del algoritmo del cálculo del error mediante el uso de scanlines, utilizando la misma herramienta ahora se suscribió al tópico de `/puzzlebot/camera/debug` en el cual se puede observar cómo el algoritmo dibuja las 5 líneas horizontales en los determinados píxeles y como se dibujan 3 puntos en la línea de la pista, dos en los bordes y una central. También indica el valor del error que se calcula dependiendo del promedio de que tan alejadas están las líneas centrales con respecto a la línea vertical que determina la mitad de la imagen.

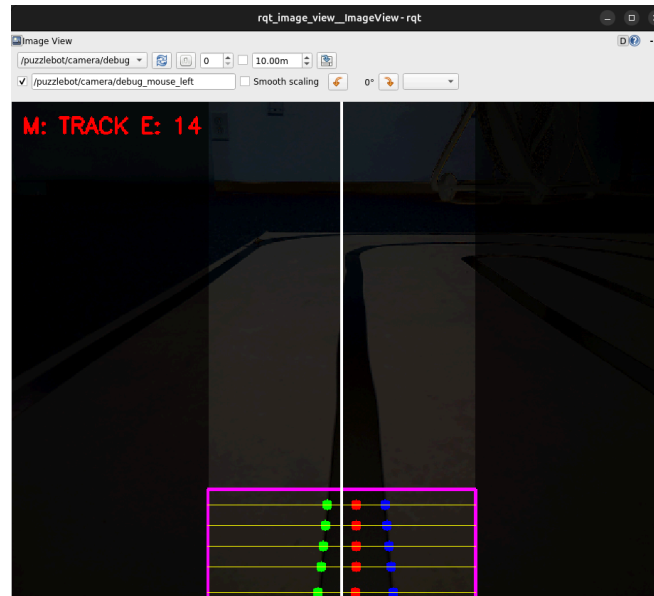


Figura 15: Scanlines y cálculo de error en línea recta.

Como se observa en la figura 15, los puntos (*scanlines*) están en una posición adecuada y el cálculo del error es el correcto ya que al ser una línea recta, los puntos centrales están casi alineados a la línea central de la imagen, por lo que el error es pequeño.

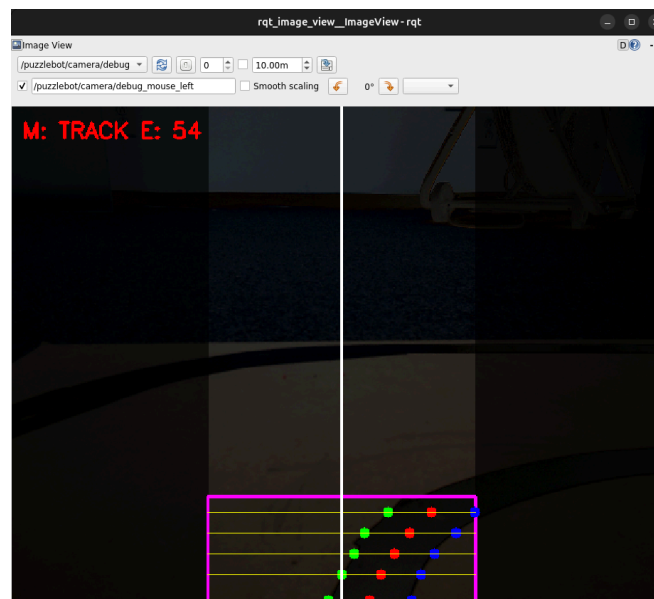


Figura 16: Scanlines y cálculo de error en una curva a la derecha.

En la figura 16, se observa cómo al entrar en una curva, los puntos centrales se desvían junto con la línea de la pista. Esto provoca que el error incremente, haciendo que el robot gire con más fuerza en dirección a lo que determine el signo del error. Se observa como los puntos en la parte inferior de la imagen están más cerca del centro de la imagen que los que están más arriba. Debido a que el error es el promedio de los puntos, esto hace que el cálculo del error sea lo más preciso posible, haciendo que el robot realice unos giros más suaves.

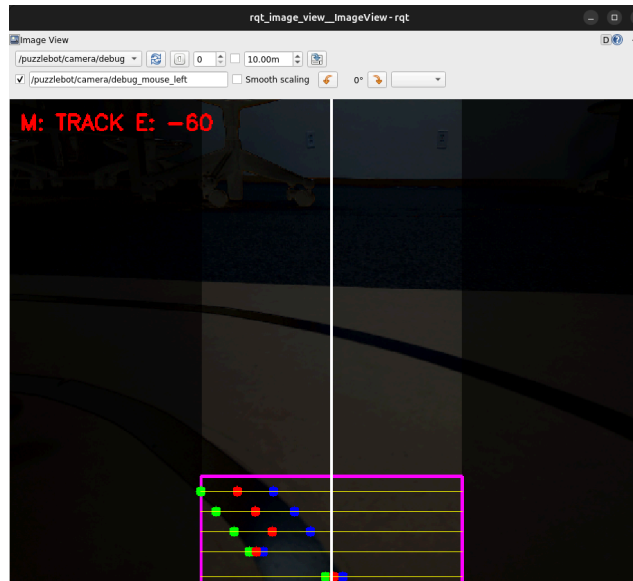


Figura 17: Scanlines y cálculo de error en curva hacia la izquierda.

Por el otro lado, como se observa en la figura 17, cuando las curvas son a la izquierda el error es negativo. También se puede observar como los centroides de las dos scanlines de la parte inferior, a pesar de que por cuestiones de ruido en la imagen, en ese frame no fueron correctamente calculados, el valor del error sigue siendo coherente debido a que también se toman en cuenta los puntos en la parte superior que si han sido calculados correctamente, determinando que el uso de varias scanlines frente a sólo un punto de referencia es mucho más robusto. De igual manera, la ROI que elimina los bordes de la imagen y los scanlines puestos lo más abajo posible de la imagen, hacen que en las curvas no tome en cuenta las líneas laterales que representan del carril, evitando así que el cálculo del error sea incorrecto.

Por último, al integrar el nodo `traffic_light_node` junto con la lógica del multiplicador de velocidad dependiendo del color detectado del semáforo se obtuvieron los siguientes resultados:

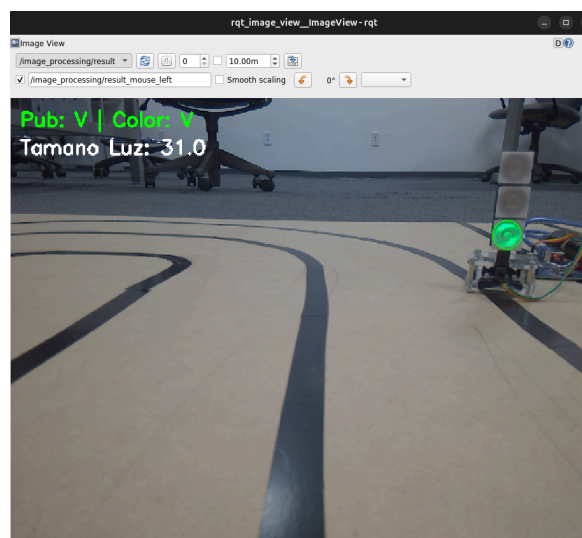


Figura 18: Detección del color verde del semáforo sobre la pista.

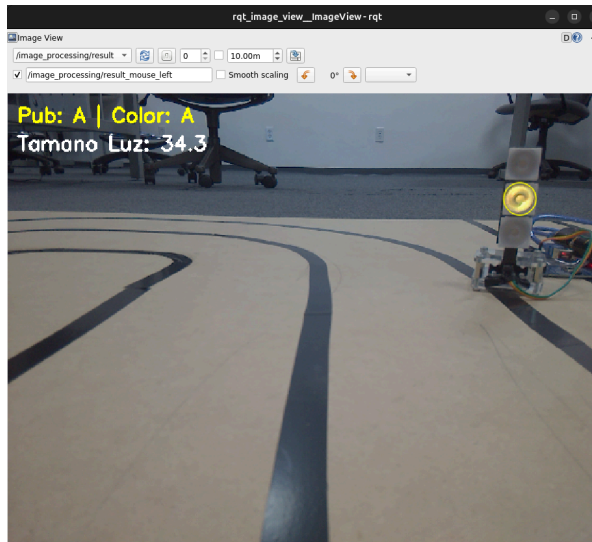


Figura 19: Detección del color amarillo del semáforo sobre la pista.

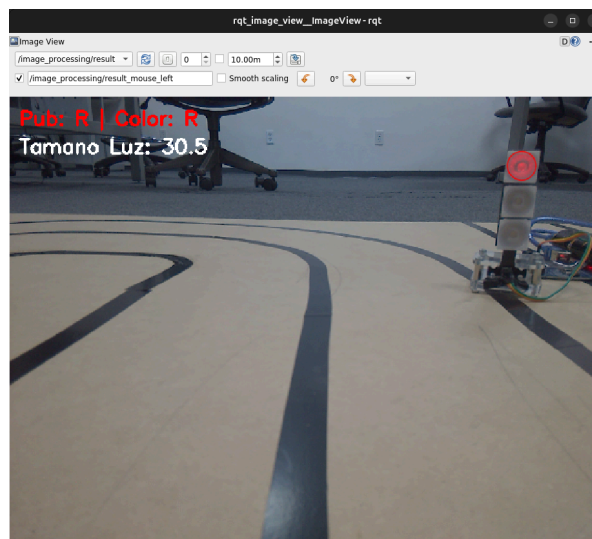


Figura 20: Detección del color rojo del semáforo sobre la pista.

Como se observa en las figuras 18, 19 y 20, los colores se detectan correctamente a una distancia de 35 cm. Esto se logra mediante la variable UMBRAL_TAMANO_LUZ_35CM, la cual se configuró en 30 puzzles para establecer la equivalencia entre el tamaño visual de la luz del semáforo y la distancia física. De este modo, el robot es capaz de reaccionar al semáforo a partir de una distancia de 35 cm.

Finalmente al ejecutar el archivo launch, se observa cómo el robot sigue la línea de manera efectiva, haciendo curvas suaves y sin oscilaciones, detectando el semáforo de manera correcta a la distancia determinada.

Asimismo se muestra un video del funcionamiento del Puzzlebot integrado con todos los elementos del reto:

<https://drive.google.com/file/d/1fFFyO4L6atiQcA6a5tukTUPjPvBrppr8/view?usp=sharing>

Conclusiones

Finalmente, se validó con éxito la eficiencia del sistema de navegación reactiva, cumpliendo los objetivos principales gracias a la implementación de un controlador PID estabilizado con un filtro EMA, la retroalimentación continua del algoritmo de scanlines, la adaptación a curvas mediante gain scheduling y la interacción del semáforo basada en el área del blob. Los objetivos de robustez absoluta se cumplieron en su totalidad debido a que en los métodos de percepción: ante cambios drásticos de luz o curvas muy pronunciadas, la binarización de Otsu no falla y en caso de que esta generará una omisión de datos, es decir, pérdida repentina de la línea, este se queda con el ángulo anterior lo cual indica robustez en el algoritmo, así mismo, se podría implementar en entregas próximas una estrategia algorítmica de recuperación, se propone integrar una máquina de estados con una rutina de barrido exploratorio para reencontrar el carril, sustituir la binarización tradicional por redes neuronales ligeras para una segmentación inmune a gradientes de iluminación, y aplicar fusión sensorial con una IMU que permita mantener la estimación de la trayectoria durante los instantes en que la cámara queda *ciega*, consolidando así un sistema verdaderamente tolerante a fallos.

Referencias

- [1] López Doicela, R. S. (2025). Estrategias de control, estimación y evasión de obstáculos para un robot móvil omnidireccional con ruedas Mecanum: Desarrollo de un controlador cinemático de seguimiento de trayectorias [Trabajo de Integración Curricular, Escuela Politécnica Nacional]. Repositorio Institucional EPN.
- [2] Sáenz, A., Santibáñez, V., Bugarín, E., Dzul, A., & Ríos, H. (2017). Control de velocidad por voltaje calculado de un robot móvil omnidireccional con realimentación visual. En Memorias del XIX Congreso Mexicano de Robótica (COMRob 2017). Mazatlán, Sinaloa, México: Asociación Mexicana de Robótica e Industria AC.
- [3] Delgado-Báez, J. A., Castro-Linares, R., & Velasco-Villa, M. (2014). Formación de robots móviles omnidireccionales considerando su modelo dinámico. En Memorias del XVI Congreso Latinoamericano de Control Automático (CLCA 2014) (pp. 14-17). Cancún, Quintana Roo, México.
- [4] Dellaert, F., & Sethi, S. (s. f.). *Differential drive robot actions*. Robotics from first principles. https://www.roboticsbook.org/S52_diffdrive_actions.html
- [5] Klančar, G., Zdešar, A., Blažič, S., & Škrjanc, I. (Eds.). (2017). *Wheeled mobile robotics: From fundamentals towards autonomous systems*. Butterworth-Heinemann
- [6] Vinke, N. (2022, 2 enero). *Image Processing on Embedded Platforms*. Medium. <https://medium.com/@narayanivinke4545/image-processing-on-embedded-platforms-a460cd6c8a04>

- [7] Boufaïda, S. O., Benmachiche, A., & Maatallah, M. (2026). Real-Time Image Processing Algorithms for Embedded Systems. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2601.06243>
- [8] Raspberry Pi Foundation. (s.f.). Getting started with the Camera Module. Raspberry Pi Projects. Recuperado el 5 de abril de 2026, de
<https://projects.raspberrypi.org/en/projects/getting-started-with-picamera/1>
- [9] Kangalaw. (2019, 2 abril). *Jetson Nano + Raspberry Pi Camera*. JetsonHacks.
<https://jetsonhacks.com/2019/04/02/jetson-nano-raspberry-pi-camera/>
- [10] Vemulapalli, K. (2025, 20 marzo). *Computer Vision*. MathWorks.
<https://la.mathworks.com/discovery/computer-vision.html>
- [11] Moukthika. (2025a, 29 abril). *Color spaces in OpenCV*. OpenCV.
<https://opencv.org/color-spaces-in-opencv/>
- [12] GeeksforGeeks. (2025, 28 abril). *Image Enhancement Techniques using OpenCV Python*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/image-enhancement-techniques-using-opencv-python/>
- [13] Trital, P. (2025, 19 mayo). *Gaussian Blur*. Medium.
https://medium.com/@prajun_t/gaussian-blur-cceb6c76a009
- [14] Moukthika. (2025b, mayo 7). *Blob Detection using OpenCV*. OpenCV.
<https://opencv.org/blob-detection-using-opencv/>
- [15] MathWorks. (s. f.). *Tipos de operaciones morfológicas*. Centro de Ayuda de MATLAB. Recuperado 10 de mayo de 2026, de
<https://la.mathworks.com/help/images/morphological-dilation-and-erosion.html>

[16] Hangrae, D. (2024, 29 de abril). Scanline Filling Algorithm. How do Computers Fill Polygons? Medium.

<https://medium.com/@dillihangrae/scanline-filling-algorithm-852ad47fb0dd>

[17] Scan Line Algorithm for Polygon Filling in Computer Graphics. (s. f.).

https://www.tutorialspoint.com/computer_graphics/computer_graphics_scan_line_algorithm.htm

[18] Computer Vision: Algorithms and Applications, 2nd ed. (s. f.).

<https://szeliski.org/Book/>

[19] Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing. Pearson.

[20] Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer vision with the OpenCV library. O'Reilly Media.

[21] Corke, P. (2023). Robotics, Vision and Control: Fundamental Algorithms in Python. Springer.